

← → ↺ 🌐 leetcode.com/problems/maximum-sum-circular-subarray/ ☆ 📁 ⬇️ 🔒 ⋮

Continue where you left off: Chrome can restore your tabs every time you restart. To turn this on, go to [Settings](#).

Problem List < > 🔍

Description Editorial Submissions Solutions

918. Maximum Sum Circular Subarray Solved ✓

Medium Topics Companies Hint

Given a **circular integer array** `nums` of length `n`, return the *maximum possible sum of a non-empty subarray of `nums`*.

A **circular array** means the end of the array connects to the beginning of the array. Formally, the next element of `nums[i]` is `nums[(i + 1) % n]` and the previous element of `nums[i]` is `nums[(i - 1 + n) % n]`.

A **subarray** may only include each element of the fixed buffer `nums` at most once. Formally, for a subarray `nums[i], nums[i + 1], ..., nums[j]`, there does not exist `i ≤ k1, k2 ≤ j` with `k1 % n == k2 % n`.

Example 1:

Input: `nums = [1,-2,3,-2]`
Output: 3
Explanation: Subarray [3] has maximum sum 3.

Example 2:

Input: `nums = [5,-3,5]`
Output: 10
Explanation: Subarray [5,5] has maximum sum 5 + 5 = 10.

Example 3:

Input: `nums = [-3,-2,-3]`
Output: -2
Explanation: Subarray [-2] has maximum sum -2.

Constraints:

7.5K 👤 141 ☆ 📁 🔒 34 Online

Code

Java 🔒 Auto

```
1 class Solution {
2
3     public int maxSubarraySumCircular(int[] nums) {
4
5         int totalSum = 0;
6
7         int currentMax = nums[0];
8         int maxSum = nums[0];
9
10        int currentMin = nums[0];
11        int minSum = nums[0];
12
13        for (int i = 0; i < nums.length; i++) {
14
15            totalSum += nums[i];
```

Saved Ln 38, Col 2

Testcase Test Result

Accepted Runtime: 0 ms

Case 1 Case 2 Case 3

Input

nums =
[1,-2,3,-2]

Output

3

Expected

3

📁 Contribute a testcase